

However, there are **five concrete production risks** that the current code does not fully address, ordered by severity:

#	Risk	Severity	Root cause
1	Main thread has no top-level watchdog — a Playwright disconnect kills the with <code>sync_playwright()</code> block and the whole process exits	High	Only inner <code>try/except</code> around <code>page.evaluate</code> , no outer supervisor around <code>main()</code> 's CDP session
2	DOM-only extraction — one Coinglass class rename = zero data across 18 assets	High	EXTRACTION_JS couples tightly to <code>[class="legend-"]</code> and <code>.includes()</code> string matches
3	Playwright JSHandle retention on 24/7 loops — <code>page.evaluate</code> returns a large nested dict every 2s, forever	Medium	No handle disposal, no periodic context recycle
4	Sync <code>page.evaluate</code> serialises 18 assets across 2 pages every 2s — ~9 round-trips through CDP	Medium	Loop calls <code>evaluate</code> per page instead of per browser
5	<code>seed_history</code> under <code>self.lock</code> performs blocking <code>requests.get()</code> calls to Binance while holding the feed lock	Medium	Lock scope too wide

Below is a concrete fix for each of the four concerns you raised, plus fix #5 as a bonus.

1. End-to-end exception handling — a supervisor around `main()`

What's wrong today: the inner loops (`tv_scraper_thread`, `WebSocket` managers) recover well, but if the *main-thread* Playwright connection dies (Chrome crash, network stack hiccup, `TargetClosedError`), the with `sync_playwright()` as `pw`: block unwinds and `main()` returns. There is no outer restart harness. The scraper thread's `while True` → with `sync_playwright()` is on the right track but the pattern is not applied to `main()`.

Fix — a supervisor wrapper. Externalise the current body of `main()` into `_run_engine_session()` and wrap it. Critically, do **not** restart on `KeyboardInterrupt` or `_SHUTDOWN_REQUESTED`.

```
import signal, traceback, time

_SHUTDOWN_REQUESTED = False

def _install_signal_handlers():
    def _handler(signum, frame):
        global _SHUTDOWN_REQUESTED
        _SHUTDOWN_REQUESTED = True
        log_message(f"[Supervisor] Signal {signum} received, requesting shutdown.")
    for s in (signal.SIGINT, signal.SIGTERM):
        try: signal.signal(s, _handler)
        except Exception: pass # SIGTERM not on Windows main thread

def supervised_main():
    _install_signal_handlers()
    backoff = 5
    MAX_BACKOFF = 300
    while not _SHUTDOWN_REQUESTED:
        start_ts = time.time()
        try:
            _run_engine_session() # <- the current body of main()
            if _SHUTDOWN_REQUESTED:
                break
            log_message(f"[Supervisor] Session returned cleanly. Restarting in 5s.")
            time.sleep(5); backoff = 5
        except KeyboardInterrupt:
            log_message(f"[Supervisor] KeyboardInterrupt. Exiting.")
            break
        except Exception as e:
            uptime = time.time() - start_ts
            log_message(f"[Supervisor] Session crashed after {uptime:.0f}s: {e}\n{traceback.format_exc()}")
            # If it crashed fast, back off harder; if it ran long, reset backoff
            if uptime > 600:
                backoff = 5
            else:
                backoff = min(backoff * 2, MAX_BACKOFF)
            log_message(f"[Supervisor] Restart in {backoff}s.")
            for _ in range(backoff):
                if _SHUTDOWN_REQUESTED: return
                time.sleep(1)
```

Additionally, inside `_run_engine_session()`, wrap `page.evaluate` with a **specific** exception classifier — the current `except Exception: page_results = {}` is too broad and hides Cloudflare re-challenges:

```
from playwright.sync_api import TimeoutError as PWTimeout, Error as PWError

def _safe_evaluate(page, js, tag=""):
    try:
        return page.evaluate(js, timeout=8000)
    except PWTimeout:
        log_message(f"[Eval:{tag}] Timeout ~ likely blocked (Cloudflare?).")
        return None
    except PWError as e:
        msg = str(e).lower()
        if "target closed" in msg or "context was destroyed" in msg:
            raise # propagate + thread reconnects
        log_message(f"[Eval:{tag}] Playwright error: {e}")
        return None
    except Exception as e:
        log_message(f"[Eval:{tag}] Unexpected: {e}")
        return None
```

The key distinction: **transient parse errors** return `None` (loop continues); **connection-level errors** re-raise so your outer `while True`: with `sync_playwright()`... block in `tv_scraper_thread` actually recycles the connection instead of silently degrading.

Finally, add a **Cloudflare captcha detector** — a passive check that runs when `all_fail` is true for `N` consecutive cycles:

```
CAPTCHA_MARKERS = ("challenge-platform", "cf-chl", "Just a moment", "Verify you are human")
```

```
def _detect_captcha(page) -> bool:
    try:
        html = page.evaluate("() -> document.documentElement.innerHTML.slice(0, 4000)")
        return any(m in html for m in CAPTCHA_ORS)
    except Exception:
        return False
```

If detected, page the user (desktop notification / webhook) rather than silently spinning — a captcha needs a human once, not more retries.

2. DOM selector brittleness — the biggest architectural risk

Your EXTRACTION_JS currently depends on three coupling points, in order of fragility:

1. `document.getElementById(tv_chart_container_win$())` — TradingView container IDs (stable-ish, but embedded)
2. `[class*="legend-"]` — TradingView legend hashed class (this is what will break first — TV routinely re-hashes CSS modules)
3. `.includes("Futures Cumulative Volume Delta")` — human-readable label match (most stable, but locale-dependent)

The good news: because your indicators come from TradingView (embedded inside the Coinglass iframe), the labels themselves ("Open Interest", "Funding Rates", "Futures Cumulative Volume Delta") are highly stable — they come from the study definitions, not from Coinglass UI. So a **label-first, class-agnostic** parser is far more resilient than what you have.

Recommended: hybrid extractor with 3 fallback tiers

```
const EXTRACTION_JS = String.raw`() => {
    const OUT = {};

    // ---- Tier helpers -----
    const clean = s => (s || "").replace(/\\u2012/g, "-").replace(/\\s+/g, " ").trim();

    // Try to find any DOM node inside a doc whose text contains `label`
    // and whose nearest number-looking sibling is the value.
    function findByLabel(doc, label) {
        // Use TreeWalker for speed and to skip script/style
        const walker = doc.createTreeWalker(doc.body, NodeFilter.SHOW_TEXT, {
            acceptNode: n => n.nodeValue && n.nodeValue.includes(label)
                ? NodeFilter.FILTER_ACCEPT : NodeFilter.FILTER_SKIP
        });
        let node;
        while ((node = walker.nextNode())) {
            // climb to the row container (legend row is typically the parent or grandparent)
            let row = node.parentElement;
            for (let hop = 0; hop < 4 && row; hop++, row = row.parentElement) {
                const txt = clean(row.innerText || "");
                // Regex tries to capture the LAST numeric token on the row
                const m = txt.match(/(-?)[\\d.,]+\\s*[KMB%]?$/);
                if (m) return m[1].replace(/\\s/g, "");
            }
        }
        return null;
    }

    // Try structured selector first (fast path), fall back to label search.
    function readIframe(iframe) {
        const doc = iframe.contentDocument || iframe.contentWindow.document;
        const data = {};

        // ---- Tier 1: symbol from title/legend header (label-based, resilient) ----
        const titleEl = doc.querySelector("[class*='title']") ||
            doc.querySelector("[data-name*='symbol']");
        data.symbol = titleEl ? clean(titleEl.innerText).split(/[/\\s/])[0] : null;

        // ---- Tier 2: close price via valueTitle 'C' (existing logic) -----
        const cEl = Array.from(doc.querySelectorAll("[class*='valueTitle-']"))
            .find(e => clean(e.innerText) === 'C');
        if (cEl) {
            const val = cEl.parentElement && cEl.parentElement.nextElementSibling;
            if (val) data.price = clean(val.innerText);
        }

        // ---- Tier 3: label-based scan (works even if legend classes change) ----
        const labelMap = {
            open_interest: "Open Interest",
            funding_rate: "Funding Rate", // matches "Funding Rate" or "Funding Rates"
            spot_cvd: "Spot Cumulative Volume Delta",
            futures_cvd: "Futures Cumulative Volume Delta",
            liquidations: "Liquidation", // matches "Liquidations"
            ls_ratio: "Long/Short", // matches "Long/Short Ratio"
            premium_index: "Premium",
            rsi: "Relative Strength"
        };
        for (const [k, lbl] of Object.entries(labelMap)) {
            if (data[k] == null) data[k] = findByLabel(doc, lbl);
        }
        return data;
    }

    for (let i = 1; i <= 9; i++) {
        const container = document.getElementById('tv_chart_container_win' + i)
        || (i === 1 ? document.getElementById('tv_chart_container_main') : null);
        const iframe = container ? container.querySelector('iframe') : null;
        if (!iframe) { OUT['win' + i] = { success: false, error: 'no-iframe' }; continue; }
        try {
            OUT['win' + i] = { success: true, data: readIframe(iframe) };
        } catch (e) {
            OUT['win' + i] = { success: false, error: String(e) };
        }
    }
    return OUT;
};`;
```

Why this survives UI churn.

- **Tier 3** walks the text tree looking for the *label string* — the labels are baked into TradingView's study code, not into Coinglass's CSS. They almost never change.
- The value is captured with a **regex on the trailing numeric token** `((-?)[\\d.,]+\\s*[KMB%]?)$`, so it works whether TV renders the number in a `<span>`, a nested `<div>`, or an SVG `<text>`.
- Individual tiers are independent: even if `valueTitle-` disappears, price falls back to a label-based scan for a candlestick close.

Should you switch to XHR/WebSocket interception?

Short answer: partially, yes — as a *shadow feed*, not a replacement.

I checked and your current code has **zero** `context.route`, `page.on("response")`, or `page.on("websocket")` handlers. Adding one for Coinglass's own network layer is the single highest-ROI resilience upgrade you can make, because it fully bypasses the DOM. The catch: Coinglass's public API is what feeds the charts, and endpoints do change. So the pragmatic pattern is:

```
# Attach ONCE per page, in the main thread after page load
def attach_network_shadow(page, shadow_store: dict, shadow_lock: threading.Lock):
    def on_response(resp):
        url = resp.url
        # Coinglass API endpoints — adjust prefixes to what you actually see in DevTools >
        # Network
        if "coinglass.com/api" not in url and "coinglass.com/public" not in url:
            return
        try:
            ct = resp.headers.get("content-type", "")
            if "json" not in ct:
                return
            body = resp.json()
        except Exception:
            return
        with shadow_lock:
            shadow_store[url] = {"ts": time.time(), "body": body}
    page.on("response", on_response)
```

Then in `get_live_data`, if a window's DOM value is `None`, look up the last matching response in `shadow_store` and use it. This gives you **defence in depth**: DOM path stays primary (you already trust it), network path stays as an always-warm backup.

Two caveats you should know before doing this:

- Coinglass's chart data endpoints are often served as **WebSocket frames**, not XHR — `page.on("websocket") + ws.on("framereceived")` may be the right hook. You will need to spend 15 min in Chrome DevTools > Network > WS on one Coinglass tab to identify the exact frame schema before wiring this up.
- Even if you use interception, **keep the DOM parser** — the DOM already gives you *rendered* values (post-processing, aggregation windows), whereas raw frames may need client-side reconstruction.

3. Thread safety & memory leaks over 24/7 runtime

Based on the actual code, here are the real leak vectors:

Vector A — history dequeues are already bounded (`maxlen=14410`). ✅ No leak here. Good design.

Vector B — no JSHandle disposal after `page.evaluate`. In your code you use `page.evaluate(EXTRACTION_JS)` which returns a serialised Python dict, **not** a JSHandle. Serialised returns are GC'd normally. However, if you ever migrate to `page.evaluate_handle(...)` (e.g. for iframe traversal), you **MUST** call `.dispose()` on the handle or Playwright's server-side references pile up. This is a landmine to flag now.

Vector C — long-lived Playwright context. The scraper thread already re-opens the context on 5 consecutive failures. Complement this with a **scheduled recycle every ~6h** to shed accumulated CDP state:

```
def tv_scraper_thread():
    while not _SHUTDOWN_REQUESTED:
        session_start = time.time()
        MAX_SESSION_SECS = 6 * 3600 # recycle every 6h even if healthy
        try:
            with sync_playwright() as pw:
                thread_browser = pw.chromium.connect_over_cdp(CDP_BASE, no_defaults=True)
                thread_scraper = CoinglassScraper()
                thread_scraper.context = thread_browser.contexts[0]
                consecutive_errors = 0
                while not _SHUTDOWN_REQUESTED:
                    if time.time() - session_start > MAX_SESSION_SECS:
                        log_message("[TVScraper] Scheduled session recycle.")
                        break
                    layout_data = thread_scraper.get_live_data()
                    if layout_data:
                        feed.update_from_layout(layout_data)
                        consecutive_errors = 0
                    else:
                        consecutive_errors += 1
                        if consecutive_errors >= 5: break
                    time.sleep(2.0)
        except Exception as e:
            log_message(f"[TVScraper] Fatal: {e}. Reconnecting in 10s.")
            time.sleep(10)
```

Vector D — the `heal_attempts` dict. I noticed you pop it on success but never garbage-collect keys that never come back (e.g. an asset permanently missing after a config change). Cap it:

```
if len(self.heal_attempts) > 200:
    self.heal_attempts.clear() # nuclear but
```

Vector E — `shadow_store` if you add network interception. Bound it: dict → `collections.OrderedDict` with manual LRU eviction at, say, 500 entries. Otherwise every response URL becomes a memory root forever.

Vector F — `seed_history` holding `self.lock` during `requests.get()`. This is a real bug (not a leak, a stall):

```
def seed_history(self, obd_cache):
    # ❌ BEFORE — blocks all readers for up to 5s * N symbols
    with self.lock:
        for symbol in TARGET_ASSETS:
            requests.get(...)
```

Fix — do I/O outside the lock:

```
def seed_history(self, obd_cache):
    # 1. Do all network I/O without the lock
    fetched = {}
    for symbol in TARGET_ASSETS:
        if symbol.endswith(("USD", "BUSD")):
            # ...
```

```
try:
    res = requests.get(
        f"https://fapi.binance.com/fapi/v1/ticker/price?symbol={symbol}",
        timeout=5)
    if res.status_code == 200:
        fetched[symbol] = float(res.json().get("price", 0.0))
    except Exception:
        fetched[symbol] = 0.0

# 2. Now take the lock and mutate
with self.lock:
    now = time.time()
    for symbol in TARGET_ASSETS:
        price_key = f"{symbol}_price_history"
        # ... rest of mutation logic using fetched[symbol]
```

4. Performance — one evaluate call across everything

You're currently paying 2 CDP round-trips per cycle (one per page). Two ways to reduce this:

Option A — merge across pages in the browser (best ROI). Attach the extractor once as a page-scoped function, and call `Promise.all` across all layout pages using a single `context.evaluate` — but Playwright doesn't expose a context-level `evaluate`. So the real win is:

Option B — batch inside each page + throttle raf. Your `EXTRACTION_JS` already returns `win1-win9` in one call, which is correct. What you can do is:

1. **Install the parser as a persistent function** using `page.add_init_script()` so it's not re-parsed each call.
2. **Use `requestAnimationFrame` inside the JS** so extraction is aligned with browser paint and doesn't fight the chart's own render loop.
3. **Return a compact schema** (arrays of numbers, not object of objects) to cut JSON serialisation cost by ~40%.

```
# Once at startup — for each layout page
INIT_SCRIPT = r"""
window.__UE3_EXTRACT = () => new Promise(resolve => {
    requestAnimationFrame(() => {
        // ... same body as EXTRACTION_JS above, but returning:
        // { s: [sym1, sym2, ...], p: [px1, px2, ...], oi: [...], fr: [...], ... }
        // Arrays of length 9, index-aligned.
        resolve(compactResult);
    });
});
"""

page.add_init_script(INIT_SCRIPT) # persists across navigation

# Per-cycle:
result = page.evaluate("(() => window.__UE3_EXTRACT())")
```

Option C — parallelise across pages with `asyncio`. Move `get_live_data` to `playwright.async_api` and gather:

```
async def get_live_data_async(self):
    pages = get_sorted_pages(self.context)
    async def one(p):
        try: return await p.evaluate("(() => window.__UE3_EXTRACT())")
        except Exception: return {}
    results = await asyncio.gather(*[one(p) for p in pages], return_exceptions=False)
    return self.merge(results)
```

Across 2 pages the wall-clock win is small (~1 RTT = 30–80ms), but at 9 assets/page × 2s cadence it removes serialisation between pages and lets Chrome parallelise the iframe reads.

Rough expected wins on the current 18-asset config:

- `add_init_script` + compact schema: ~30–50% lower payload size, ~10–20ms saved per cycle.
- Async gather across pages: another ~50–80ms.
- Combined: cycle time from ~200–300ms → ~80–120ms, which materially improves the freshness of your 15-min Z-scores at the edges of each bar.

Anti-patterns worth flagging (not critical, but worth cleaning up)

1. **`import msvcrt` at module top level** hard-locks the engine to Windows. If you ever run this on a Linux VPS for 24/7 uptime, use a guarded import + `select`-based `stdin` fallback.
2. **`print()` calls in error paths** (e.g. `[FootprintScraper] Connection error...`) — everything should go through `log_message()` so it's captured in your log queue and doesn't fight the Rich Live view.
3. **`while True` inside `_listen` without a shutdown flag check** — `WebSocket` workers should observe `_SHUTDOWN_REQUESTED` so `Ctrl+C` is clean.
4. **`self.heal_attempts` state on a scraper that gets rebuilt** — since `tv_scraper_thread` re-creates `CoinGlassScraper()` on reconnect, healing state is lost across reconnects. Move it to the feed if you want it to survive reconnects.
5. **Two-tab hard-coding.** `PAGE_ASSETS[page_idx]` assumes ordering matches the CDP page listing. Add an explicit `data-ue3-page-idx` attribute injected via `page.evaluate("document.title = ...")` and match by title, not by list order — otherwise a tab reorder silently corrupts the asset→symbol mapping.

Priority-ordered action list

1. **Ship the supervisor wrapper (§1)** — 1 hour, eliminates the most likely 24/7 crash mode.
2. **Fix `seed_history` lock scope (§3 vector F)** — 15 minutes, removes a real stall.
3. **Deploy the label-based extractor tier (§2)** — half a day, buys you months of runway against Coinglass/TV UI churn.
4. **Add scheduled 6h context recycle + `heal_attempts` cap (§3)** — 30 minutes.
5. **Install `add_init_script` + compact schema (§4)** — a few hours, real latency win.
6. **Attach `page.on("response")` shadow feed (§2 addendum)** — a day, but treat it as belt-and-braces, not a rewrite.

If you'd like, I can turn any one of these into a full drop-in patch against your current file — say the word and point me at the target section.

Super Agent

Why pick ChatGPT or Claude? Both. →

Ask anything, create anything

+

Ultra ▾

Connectors

Speak